



**UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO**

FACULTAD DE INGENIERÍA

SISTEMAS DISTRIBUIDOS

ING. OSCAR MANUEL RUIZ HURTADO

Nombre:

Velasco Pacheco Javier

Bitacora - S07

**Mexi Banco: de un monolito a servicios que
se encuentran solos**



Ciudad de México, México

09 de octubre de 2025

Paso 1. La historia: partimos el banco

Al analizar la evolución de Mexi Banco, comprendí que uno de los principales cambios al pasar de un monolito a una arquitectura distribuida fue la forma en que se comunican los componentes. Antes, una llamada a método permitía ejecutar una función directamente dentro de la misma aplicación. En cambio, al separar las funciones en servicios independientes, la comunicación comenzó a realizarse mediante peticiones HTTP, por lo que ahora depende de la red y de que el servicio solicitado esté disponible. Esto significa que pueden aparecer errores de conexión, tiempos de espera o respuestas inesperadas. Además, si una operación involucra varios servicios, ya no se puede asumir que todos los cambios se desharán automáticamente cuando uno de ellos falle. Entendí que separar una aplicación en servicios facilita su mantenimiento y escalamiento, pero también obliga a considerar los problemas de comunicación y recuperación ante errores.

Paso 2. El directorio: que cada servicio se anote solo

2.1. Estado inicial del directorio

Al iniciar esta parte de la práctica, levanté los servicios de Mexi Banco y accedí al puerto 8761 para revisar el tablero de Eureka. En ese momento, el directorio todavía no estaba disponible, por lo que no podía consultar el registro de los servicios desde la interfaz web. Esta primera comprobación me permitió identificar el estado inicial del sistema y tener un punto de comparación para verificar el resultado después de configurar el directorio.

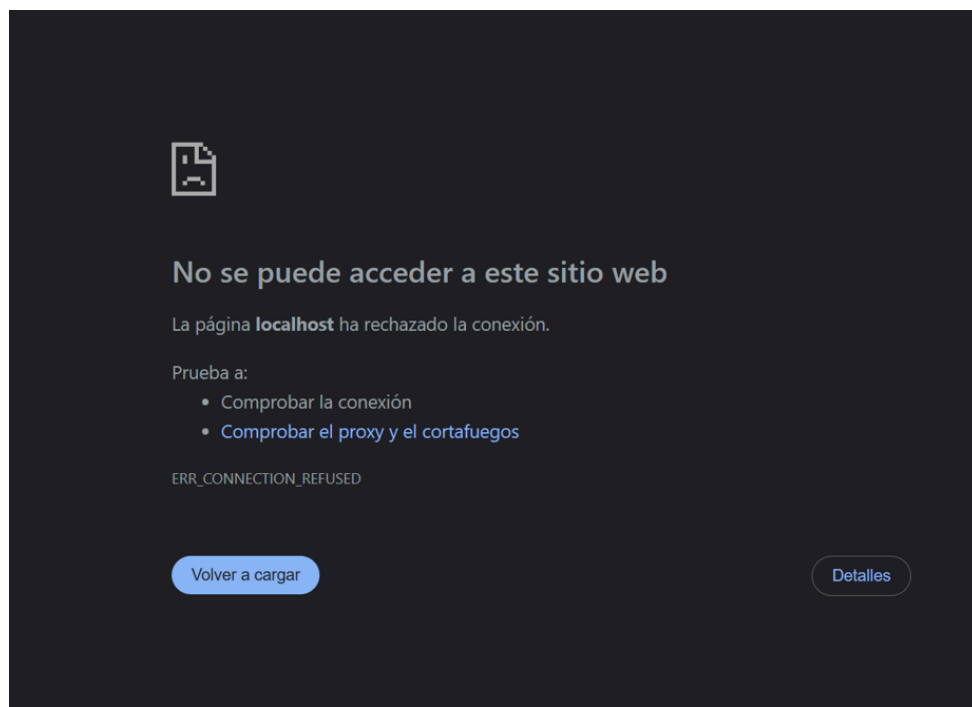
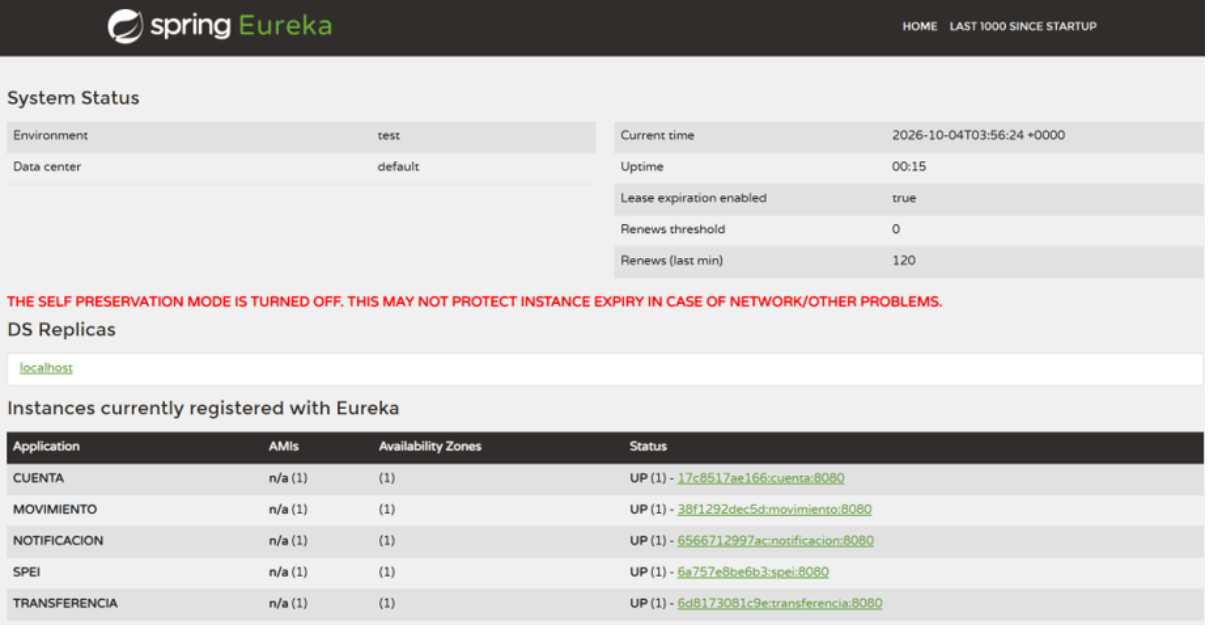


Figura 2. Estado inicial del directorio Eureka.

2.2. Configuración y registro de los servicios

Para habilitar el directorio, configuré el servidor Eureka y los clientes de descubrimiento en los servicios correspondientes. Después de realizar los cambios en el código y en los archivos de configuración, reconstruí y levanté los contenedores para aplicar la nueva configuración. Posteriormente, accedí nuevamente al tablero de Eureka y comprobé que aparecieran los cinco servicios principales: CUENTA, MOVIMIENTO, NOTIFICACION, SPEI y TRANSFERENCIA, todos con estado UP. Esto confirmó que los servicios se habían registrado correctamente y que el directorio podía reconocer sus instancias activas.



The screenshot shows the Spring Eureka dashboard. At the top, there's a header with the Spring Eureka logo and navigation links for HOME and LAST 1000 SINCE STARTUP. Below the header, the 'System Status' section displays various metrics in a table format. A red warning message indicates that the self-preservation mode is turned off. The 'DS Replicas' section shows 'localhost'. The 'Instances currently registered with Eureka' section contains a table with five rows, each representing a service: CUENTA, MOVIMIENTO, NOTIFICACION, SPEI, and TRANSFERENCIA. Each row lists the application name, the number of instances (n/a (1)), the availability zones ((1)), and the status (UP (1)) along with a unique instance ID.

Application	AMIs	Availability Zones	Status
CUENTA	n/a (1)	(1)	UP (1) - 17c8517ae166:cuenta:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 38f1292dec5d:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 6566712997ac:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 6a757e8be6b3:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 6d8173081c9:transferencia:8080

Figura 3. Los cinco servicios registrados en Eureka.

2.3. Consulta del registro XML de CUENTA

También consulté el endpoint `http://localhost:8761/eureka/apps/CUENTA` para revisar la información que Eureka proporciona sobre el servicio de cuentas. La respuesta XML permite observar los datos de la aplicación registrada y de su instancia, incluyendo información como su nombre, estado y dirección de acceso. Esta comprobación complementa la información del tablero, ya que permite consultar el registro mediante una respuesta que puede utilizarse desde otras aplicaciones.

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
▼<application>
  <name>CUENTA</name>
  ▼<instance>
    <instanceId>17c8517ae166:cuenta:8080</instanceId>
    <hostName>172.22.0.4</hostName>
    <app>CUENTA</app>
    <ipAddr>172.22.0.4</ipAddr>
    <status>UP</status>
    <overriddenstatus>UNKNOWN</overriddenstatus>
    <port enabled="true">8080</port>
    <securePort enabled="false">443</securePort>
    <countryId>1</countryId>
    ▼<dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
      <name>MyOwn</name>
    </dataCenterInfo>
    ▼<leaseInfo>
      <renewalIntervalInSecs>5</renewalIntervalInSecs>
      <durationInSecs>15</durationInSecs>
      <registrationTimestamp>1791085260748</registrationTimestamp>
      <lastRenewalTimestamp>1791086226778</lastRenewalTimestamp>
      <evictionTimestamp>0</evictionTimestamp>
      <serviceUpTimestamp>1791085259909</serviceUpTimestamp>
    </leaseInfo>
    ▼<metadata>
      <management.port>8080</management.port>
    </metadata>
    <homePageUrl>http://172.22.0.4:8080</homePageUrl>
    <statusPageUrl>http://172.22.0.4:8080/actuator/info</statusPageUrl>
    <healthCheckUrl>http://172.22.0.4:8080/actuator/health</healthCheckUrl>
    <vipAddress>cuenta</vipAddress>
    <secureVipAddress>cuenta</secureVipAddress>
    <isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
    <lastUpdatedTimestamp>1791085260748</lastUpdatedTimestamp>
    <lastDirtyTimestamp>1791085259487</lastDirtyTimestamp>
    <actionType>ADDED</actionType>
  </instance>
</application>
```

Figura 4. Información de CUENTA en el registro de Eureka.

2.4. Piensa:

Aunque los cinco servicios ya se registraron exitosamente en Eureka y aparecen como UP en el tablero, la comunicación entre ellos sigue siendo estática. El servicio de transferencia sigue resolviendo la ubicación mediante el DNS interno de Docker gracias a la variable de entorno hardcodeda (CUENTA_URL: http://cuenta:8080) en el docker-compose.yml.

El directorio resuelve el problema de localizar los servicios y conocer cuáles están disponibles sin tener que escribir manualmente la dirección de cada instancia. Si una dirección cambia o se agregan nuevas réplicas, mantener toda esa información de manera manual puede complicar la comunicación entre los componentes. Con Eureka, cada servicio puede registrarse y los demás componentes pueden consultar el directorio para encontrar las instancias disponibles. De esta forma, la comunicación entre servicios se vuelve más flexible y se facilita el escalamiento de la aplicación.

Paso 3. Gateway: una sola puerta

Para comprobar el funcionamiento del gateway, realicé diferentes peticiones y comparé las respuestas obtenidas. Primero, al intentar registrar un movimiento directamente mediante el puerto 8082, recibí una respuesta HTTP 400. Después, al consultar una cuenta inexistente a través del gateway, recibí un 404, porque la petición llegó al servicio de cuentas, pero el recurso solicitado no existía. En cambio, cuando intenté registrar un movimiento mediante el gateway, también recibí un 404, pero por una razón diferente: la ruta no estaba configurada para aceptar esa operación, por lo que el gateway no encontró una ruta que coincidiera con la petición. Finalmente, comprobé el funcionamiento de las operaciones permitidas al crear una cuenta, que devolvió 201 Created, y consultar posteriormente esa misma cuenta, que devolvió 200 OK.

```
D:\mexi-banco>curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
Content-Type: application/problem+json
Transfer-Encoding: chunked
Date: Sun, 04 Oct 2026 03:58:56 GMT
Connection: close

{"type":"about:blank","title":"Bad Request","status":400,"detail":"Failed to read request","instance":"/movimientos"}
D:\mexi-banco>
```

Figura 5. Respuesta HTTP 400 al intentar registrar un movimiento directamente en el puerto 8082.

```
D:\mexi-banco>curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 17c8517ae166
Content-Type: application/problem+json
Date: Sun, 04 Oct 2026 04:24:00 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
```

Figura 6. Respuesta HTTP 404 al consultar una cuenta inexistente mediante el gateway. Incluye la cabecera X-Instancia si aparece.

```
D:\mexi-banco>curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 130

{"timestamp":"2026-10-04T04:24:31.756+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"34147647-63"}
D:\mexi-banco>
```

Figura 7. Respuesta HTTP 404 al intentar registrar un movimiento mediante el gateway.

```
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: e49702aba78c
Content-Type: application/json
Date: Sun, 04 Oct 2026 05:05:33 GMT
```

Figura 8. Respuesta HTTP 201 Created al crear una cuenta.

```
D:\mexi-banco>curl -i http://localhost:8080/api/cuentas/%LAB_CLABE%
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 17c8517ae166
Content-Type: application/json
Date: Sun, 04 Oct 2026 04:30:19 GMT

{"id":1,"clabe":"S07809331633","titular":"S07","saldo":0.00}
```

Figura 9. Respuesta HTTP 200 OK al consultar la cuenta creada.

Pasos 4 y 5. Copias y caída

4.1. Configuración de tres réplicas

En esta parte modifiqué la configuración de Docker para ejecutar tres réplicas del servicio CUENTA. Después de reconstruir y levantar los contenedores, revisé el tablero de Eureka para comprobar que el servicio apareciera con tres instancias en estado UP. Esto permitió verificar que el sistema podía mantener varias instancias del mismo servicio registradas al mismo tiempo. La ventaja de este enfoque es que las peticiones pueden distribuirse entre distintas réplicas, en lugar de depender de una única instancia para atender todas las solicitudes.

```
D:\mexi-banco>docker ps --filter "name=cuenta" --format "{{.ID}} {{.Names}} {{.Ports}}"
99d7cd9c4972 mexi-banco-v06-cuenta-3 8080/tcp
2068c0089a37 mexi-banco-v06-cuenta-2 8080/tcp
990c13586f80 mexi-banco-v06-cuenta-1 8080/tcp
```

Figura 10. Tres instancias del servicio CUENTA registradas en Eureka.

4.2. Prueba de balanceo con nueve peticiones

Para comprobar el balanceo de carga, realicé nueve consultas consecutivas y revisé la cabecera X-Instancia de cada respuesta. Esta cabecera permitió identificar cuál de las réplicas había atendido cada petición y observar cómo se distribuían las solicitudes entre las instancias disponibles. Con esta prueba entendí que el balanceador permite repartir las peticiones sin que el cliente tenga que seleccionar manualmente qué contenedor debe atenderlas.

```
X-Instancia: 377d93cfcae2
X-Instancia: b631c7e9af8d
X-Instancia: a38c63d9b56f
X-Instancia: 377d93cfcae2
X-Instancia: b631c7e9af8d
X-Instancia: a38c63d9b56f
X-Instancia: 377d93cfcae2
X-Instancia: b631c7e9af8d
X-Instancia: a38c63d9b56f
```

Figura 11. Nueve peticiones.

4.3. Mi predicción antes de detener una réplica

Antes de detener una réplica, pensé que algunas peticiones podrían fallar durante unos momentos mientras el sistema detectaba que una instancia ya no estaba disponible. También esperaba que las otras dos réplicas continuaran respondiendo y que el balanceador dejara de enviarles solicitudes a la instancia caída después de un tiempo. No estaba seguro de si el cambio sería inmediato o si se presentarían algunos errores antes de que el sistema se estabilizara.

5.1. Hora de apagado y salida del directorio

- Hora del apagado: 22:44:54
- Primer error HTTP 500: 22:45:35
- Tiempo hasta el primer error: 41 segundos
- Último error HTTP 500: 22:46:02
- Tiempo hasta el último error: 68 segundos
- Total de errores HTTP 500: 6
- Comportamiento posterior: las solicitudes restantes respondieron con HTTP 200.

```
done
22:44:40 200
22:44:42 200
22:44:44 200
22:44:46 200
22:44:47 200
22:44:49 200
22:44:51 200
22:44:53 200
22:44:55 200
22:44:56 200
22:44:58 200
22:45:00 200
22:45:01 200
22:45:04 200
22:45:06 200
22:45:08 200
22:45:10 200
22:45:12 200
22:45:13 200
22:45:15 200
22:45:17 200
22:45:19 200
22:45:21 200
22:45:22 200
22:45:24 200
22:45:26 200
22:45:28 200
22:45:29 200
22:45:31 200
22:45:33 200
22:45:35 500
22:45:39 500
22:45:43 200
22:45:44 500
22:45:48 200
22:45:49 500
22:45:54 200
22:45:56 500
22:45:59 200
22:46:01 200
22:46:02 500
22:46:07 200
22:46:11 200
22:46:12 200
22:46:14 200
22:46:16 200
22:46:17 200
22:46:19 200
22:46:21 200
22:46:22 200
22:46:24 200
22:46:26 200
22:46:28 200
22:46:29 200
22:46:31 200
22:46:32 200
22:46:34 200
```

Figura 12. Detención de una réplica de CUENTA.

Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
CUENTA	n/a (2)	(2)	UP (2) - 2068c0089a37:cuenta:8080 , 99d7cd9c4972:cuenta:8080

Figura 13. Eureka después de detectar la caída de una instancia.

5.2. Número de errores y comportamiento del sistema

En mi prueba observé lo siguiente:

Número total de errores: 6.

Códigos HTTP que aparecieron: HTTP 200 y HTTP 500.

Fueron intermitentes, ya que algunas peticiones devolvieron HTTP 500, mientras que otras continuaron respondiendo correctamente con HTTP 200.

Al quedar dos instancias disponibles, el servicio pudo continuar atendiendo las solicitudes mediante las réplicas restantes. En mi prueba, después del último error registrado a las 22:46:02, las peticiones restantes respondieron con HTTP 200. Esto muestra que el sistema pudo seguir funcionando a pesar de la caída de una réplica.

```

22:45:44 500
22:45:48 200
22:45:49 500
22:45:54 200
22:45:56 500
22:45:59 200
22:46:01 200

```

Figura 14. Errores observados durante la caída de una réplica.

Cierre

El directorio resuelve el problema de saber qué servicios están disponibles y dónde encontrarlos sin tener que escribir sus direcciones manualmente. **El gateway resuelve** el acceso a los servicios mediante una sola entrada y permite controlar qué rutas están disponibles desde afuera. **El balanceador resuelve** el problema de repartir las peticiones entre varias copias de un servicio. **Cuando murió una copia** vi errores HTTP 500 intermitentes **porque** el gateway todavía podía enviar peticiones a la instancia que ya no estaba funcionando, hasta que se actualizó la información de los servicios disponibles.

Tarea

Para finalizar, ejecuté la demostración de la versión v06 y revisé los resultados de las operaciones de transferencia y SPEI. En esta prueba es importante comprobar tanto las respuestas de las operaciones como los saldos finales de las cuentas, ya

que permiten verificar si los movimientos se realizaron de manera correcta. La captura de la demostración sirve como evidencia del comportamiento del sistema y permite relacionar los resultados descritos con la ejecución real del programa.

```
== 5. Saldos: el dinero no se crea ni se pierde ==
$ curl http://localhost:8080/api/cuentas/002180009160699201
{"id":7,"clabe":"002180009160699201","titular":"Ana","saldo":800.00}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009160699202
{"id":8,"clabe":"002180009160699202","titular":"Beto","saldo":700.00}
[HTTP 200]
-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700
```

Figura 15. Resultados de la demostración v06 y saldos finales de las cuentas.

El resultado de a salida de bash demo-v06.sh

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco (trabajo-s07)
$ bash demo-v06.sh --latido

=== 0. Esperar a que el directorio conozca a todos y la puerta encuentre a
cuenta ===
-> listo en 2 s. Lo que sabe el directorio (http://localhost:8761/eureka/apps):
CUENTA      3 instancia(s)
GATEWAY      1 instancia(s)
MOVIMIENTO   1 instancia(s)
NOTIFICACION 1 instancia(s)
SPEI         1 instancia(s)
TRANSFERENCIA 1 instancia(s)
-> 404 de cuenta (no 503 del gateway) para una CLABE que no existe: la
puerta ya sabe llegar

=== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas)
===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type:
application/json' -d
'{"clabe":"002180009160699201","titular":"Ana","saldoInicial":1000}'
{"id":7,"clabe":"002180009160699201","titular":"Ana","saldo":1000}
[HTTP 201]
-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas

=== 2. Abrir la cuenta de Beto con 500 (misma puerta) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type:
application/json' -d
'{"clabe":"002180009160699202","titular":"Beto","saldoInicial":500}'
{"id":8,"clabe":"002180009160699202","titular":"Beto","saldo":500}
[HTTP 201]

=== 3. Transferir 200 de Ana a Beto (misma puerta: POST
/api/transferencias) ===
```

```
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160699201","claveDestino":"002180009160699202","monto":200}'
```

```
{ "id":3,"claveOrigen":"002180009160699201","claveDestino":"002180009160699202","monto":200,"fecha":"2026-10-10T04:36:36.488257746Z" }
```

```
[HTTP 201]
```

-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:

cuenta-1 GET /cuentas/002180009160699201 atendida por 377d93cfcae2

cuenta-3 GET /cuentas/002180009160699202 atendida por b631c7e9af8d

cuenta-2 POST /cuentas/002180009160699201/cargos atendida por a38c63d9b56f

cuenta-1 POST /cuentas/002180009160699202/abonos atendida por 377d93cfcae2

-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

=== 4. Consultar esa transferencia ===

```
$ curl http://localhost:8080/api/transferencias/3
```

```
{ "id":3,"claveOrigen":"002180009160699201","claveDestino":"002180009160699202","monto":200.00,"fecha":"2026-10-10T04:36:36.488258Z" }
```

```
[HTTP 200]
```

=== 5. Saldo: el dinero no se crea ni se pierde ===

```
$ curl http://localhost:8080/api/cuentas/002180009160699201
```

```
{ "id":7,"clabe":"002180009160699201","titular":"Ana","saldo":800.00 }
```

```
[HTTP 200]
```

```
$ curl http://localhost:8080/api/cuentas/002180009160699202
```

```
{ "id":8,"clabe":"002180009160699202","titular":"Beto","saldo":700.00 }
```

```
[HTTP 200]
```

-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500).
Esperado: 800 y 700

=== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ===

```
$ curl http://localhost:8080/api/movimientos?clabe=002180009160699201
```

```
[{"id":13,"clabeCuenta":"002180009160699201","tipo":"TRANSFERENCIA_ENVIADA","monto":-200.00,"saldoResultante":800.00,"fecha":"2026-10-10T04:36:36.462693Z","referencia":"a002180009160699202"}, {"id":11,"clabeCuenta":"002180009160699201","tipo":"DEPOSITO","monto":1000.00,"saldoResultante":1000.00,"fecha":"2026-10-10T04:36:34.731751Z","referencia":"saldo inicial"}]
```

```
[HTTP 200]
```

```
$ curl http://localhost:8080/api/notificaciones?clabe=002180009160699202
```

```
[{"id":8,"clabeCuenta":"002180009160699202","mensaje":"TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.00","fecha":"2026-10-10T04:36:36.569705Z"}]
```

```
[HTTP 200]
```

```

=== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ===
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type:
application/json' -H 'Idempotency-Key: spei-demo-91606992' -d
'{"claveOrigen":"002180009160699201","bancoDestino":"BANCO-X","claveDe
stino":"012180000000000009","monto":50}'
{"id":5,"idempotencyKey":"spei-demo-91606992","claveOrigen":"00218000916
0699201","bancoDestino":"BANCO-X","claveDestino":"012180000000000009
","monto":50,"estado":"ENVIADO"}
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type:
application/json' -H 'Idempotency-Key: spei-demo-91606992' -d
'{"claveOrigen":"002180009160699201","bancoDestino":"BANCO-X","claveDe
stino":"012180000000000009","monto":50}'
{"id":5,"idempotencyKey":"spei-demo-91606992","claveOrigen":"00218000916
0699201","bancoDestino":"BANCO-X","claveDestino":"012180000000000009
","monto":50.00,"estado":"ENVIADO"}
[HTTP 201]
-> mismo id (5 -> 5): no se cobro dos veces. Saldo de Ana: 750.00 (esperado
750)

```

```

=== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma
puerta ===

```

```

consulta 1 X-Instancia: 377d93cfcae2 (mexi-banco-v06-cuenta-1)
consulta 2 X-Instancia: b631c7e9af8d (mexi-banco-v06-cuenta-3)
consulta 3 X-Instancia: 377d93cfcae2 (mexi-banco-v06-cuenta-1)
consulta 4 X-Instancia: a38c63d9b56f (mexi-banco-v06-cuenta-2)
consulta 5 X-Instancia: b631c7e9af8d (mexi-banco-v06-cuenta-3)
consulta 6 X-Instancia: 377d93cfcae2 (mexi-banco-v06-cuenta-1)
consulta 7 X-Instancia: a38c63d9b56f (mexi-banco-v06-cuenta-2)
consulta 8 X-Instancia: b631c7e9af8d (mexi-banco-v06-cuenta-3)
consulta 9 X-Instancia: b631c7e9af8d (mexi-banco-v06-cuenta-3)
-> 3 replicas distintas en 9 consultas; 1 veces se repitio la misma dos veces
seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el
8080

```

```

=== 9. Los endpoints internos no tienen puerta ===

```

```

$ curl -X POST http://localhost:8080/api/cuentas/002180009160699201/abonos -H
'Content-Type: application/json' -d '{"monto":1000000}'
{"timestamp":"2026-10-10T04:36:50.896+00:00","path":"/api/cuentas/0021800
09160699201/abonos","status":404,"error":"Not
Found","requestId":"73251e9d-482"}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red
siguen abiertos (seguridad: mas adelante)

```

```

=== 10. Errores de negocio, por la puerta, con el mismo codigo que dio

```

```

cuenta ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type:
application/json' -d
'{"claveOrigen":"002180009160699201","claveDestino":"00218000916069929
9","monto":10}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la
CLABE
002180009160699299","instance":"/transferencias","servicio":"cuenta"}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type:
application/json' -d
'{"claveOrigen":"002180009160699201","claveDestino":"00218000916069920
2","monto":99999}'
{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Saldo
insuficiente en la CLABE
002180009160699201","instance":"/transferencias","servicio":"cuenta"}
[HTTP 422]

```

=== 11. Los saldos no se movieron con los errores ===

-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

=== 12. El latido: matar una replica de cuenta sin que se despida ===

-> en el directorio hay 3 instancias de CUENTA; la victima es b631c7e9af8d (mexi-banco-v06-cuenta-3)

-> docker kill = se va la luz: el proceso no alcanza a darse de baja. Solo dejan de llegar sus latidos.

-> (docker stop seria un apagado ordenado: Spring la marca DOWN al instante y nadie le manda trafico)

t= 0s CUENTA en el directorio: 3 por la puerta: 1 bien, 0 con error

t= 10s CUENTA en el directorio: 3 por la puerta: 4 bien, 2 con error

-> el directorio saco a la replica muerta a los 33 s

-> mientras tanto, por la puerta: 9 respuestas bien y 5 con error (el ultimo error a los 28 s)

-> los errores son el gateway mandando peticiones a una replica que ya no existe: el registro es eventualmente consistente

Lo que sabe ahora el directorio:

CUENTA 2 instancia(s)

GATEWAY 1 instancia(s)

MOVIMIENTO 1 instancia(s)

NOTIFICACION 1 instancia(s)

SPEI 1 instancia(s)

TRANSFERENCIA 1 instancia(s)

=== 13. Volver a prender la replica y ver cuanto tarda en reaparecer ===

-> mexi-banco-v06-cuenta-3 aparece otra vez en el directorio a los 18 s de docker start (arranque de la JVM + registro)

-> y el gateway le vuelve a mandar peticiones a los 23 s

Fin. Con BALANCEADOR_CACHE=5s EUREKA_EXPIRA_S=15
EUREKA_LATIDO_S=5 EUREKA_CONSULTA_S=5 (Spring Cloud trae latido
30, expiracion 90, consulta 30 y cache 35s).